



ΣΥΣΤΗΜΑΤΑ ΔΙΑΧΕΙΡΙΣΗΣ ΚΑΙ ΑΝΑΛΥΣΗΣ ΔΕΔΟΜΕΝΩΝ

ΕΑΡΙΝΟ ΕΞΑΜΗΝΟ 2023-2024

ΓΙΩΡΓΟΣ ΚΟΥΡΟΣ

ΑΜ : 3190095

ΠΡΩΤΗ ΕΡΓΑΣΙΑ



Ζήτημα 1

```
checkpoint
dbcc dropcleanbuffers
set statistics io on
set statistics time on
select displayName, profileViews
from users
where year(CreationDate) = 2010
order by CreationDate, profileViews;
set statistics time off
set statistics io off

checkpoint
dbcc dropcleanbuffers
create nonclustered index idx1 on users(CreationDate, profileViews) include (displayname)
--nonclustered αφού xreiazomaste sygkekrimena pedia tou pinaka users kai oxi olokliro opote mas symferei na kanoume separate ta data pou theloume na mas dosei to
--query, wste na anaktithoun grigorotera apo ton server xoris na metaboume ston arxiko pinaka users
set statistics io on
set statistics time on
select displayName, profileViews
from users
where CreationDate >= '2010-01-01' AND CreationDate < '2011-01-01'
order by CreationDate, profileViews;
set statistics time off
set statistics io off
drop index idx1 on users
```

Αρχικά αφαιρέθηκε η συνάρτηση year() ώστε να αποφύγουμε τους υπολογισμούς που θα επιβαρύνουν το query μας ώστε να βελτιωθεί η απόδοση και να μπορεί να εφαρμοστούν στο query ευρετήρια στο column CreationDate. Δημιουργήθηκε στα columns CreationDate και profileViews το ευρετήριο idx1 αφού έχουμε ταξινόμηση βάση αυτών των 2 column και επιπρόσθετα έγινε Include του displayName ώστε ο sql server να ανακτά όλα τα απαραίτητα δεδομένα απευθείας από το ευρετήριο χωρίς να ανατρέχει στον ίδιο τον πίνακα Users βελτιώνοντας έτσι σημαντικά την απόδοση μειώνοντας κατά πολύ τα logical reads (statistics io), cpu time (statistics time) όπως φαίνεται στα παρακάτω στιγμιότυπα οθόνης.

```
(199550 rows affected)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page
Table 'Users'. Scan count 13, logical reads 6001, physical reads 2, page server reads 0, read-ahead reads 5717, page

(1 row affected)

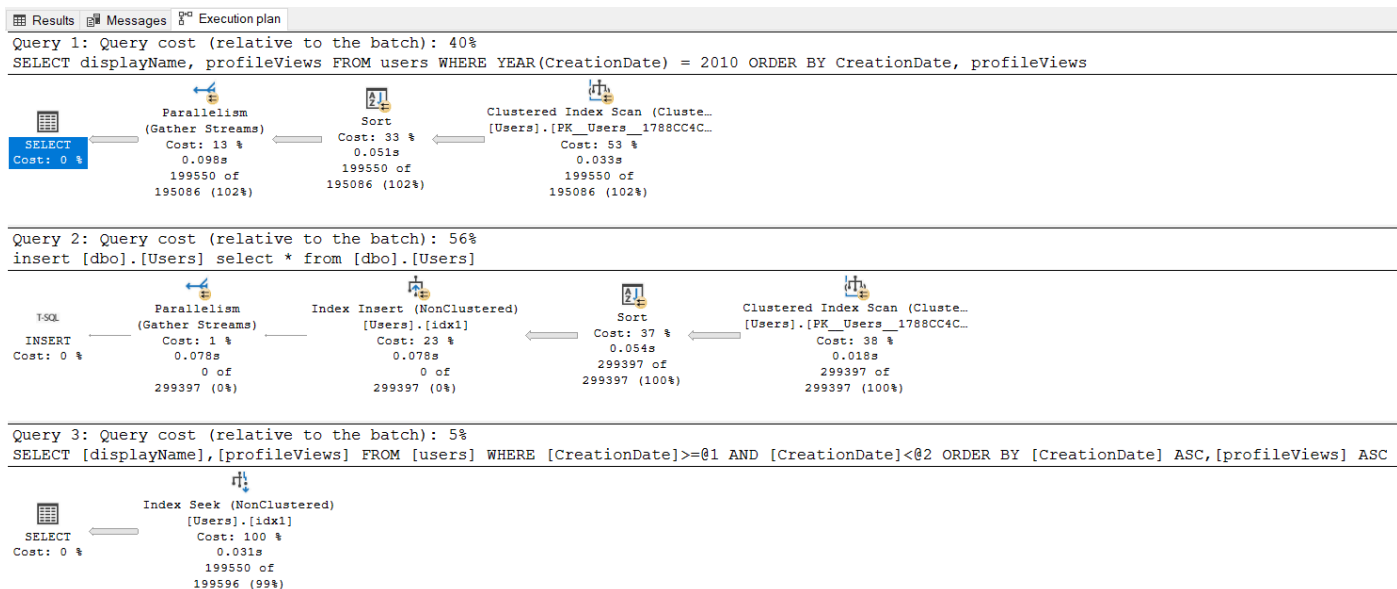
SQL Server Execution Times:
    CPU time = 344 ms,  elapsed time = 1026 ms.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
SQL Server parse and compile time:
    CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
    CPU time = 0 ms, elapsed time = 1 ms.
SQL Server parse and compile time:
    CPU time = 0 ms, elapsed time = 0 ms.

(199550 rows affected)
Table 'Users'. Scan count 1, logical reads 933, physical reads 0, page server reads 0, read-ahead reads 49, page

(1 row affected)

SQL Server Execution Times:
    CPU time = 47 ms,  elapsed time = 995 ms.
```

Συνολικά η μείωση του κόστους του query (query 3) με το index φαίνεται και από το execution plan της επόμενης σελίδας, όπου έχει μειωθεί, από 40% σε 5%.

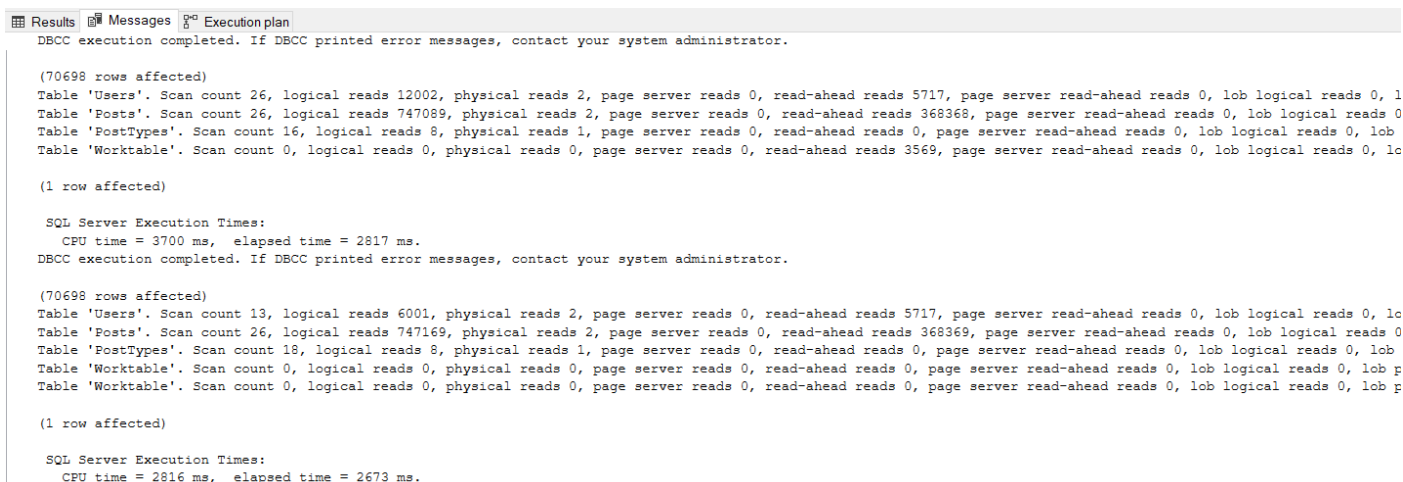


Ζήτημα 2

Αρχικά βελτιώνουμε το query χρησιμοποιώντας exist και not exist επιλέγοντας μόνο ένα post ‘Answer’ και ‘Question’ αντί να ανακτούμε περισσότερα δεδομένα και επιβαρύνουμε τον sql server αφού μας ενδιαφέρει κάθε χρήστης να έχει τουλάχιστον ένα post απάντησης και με το που βρει η exist και η not exist μια απάντηση θα σταματήσει να ανακτά περισσότερα δεδομένα. Αντίστοιχα και για τα question post, αν βρεθεί ένα τουλάχιστον δεν θα προστεθεί στα αποτελέσματα ο συγκεκριμένος χρήστης. Αυτό μας επιφέρει μια μικρή βελτίωση του elapsed time αλλά μια σημαντική βελτίωση του cpu time καθώς μειώνουμε κατά 50% τα logical reads στον πίνακα Users. Επόμενο βήμα η εφαρμογή ευρετηρίου/ων.

```
checkpoint
dbcc dropcleanbuffers
set statistics io on
set statistics time on
SELECT users.* FROM users, posts, postTypes WHERE users.userid=posts.ownerUserId and posts.postTypeId=PostTypes.postTypeId and postTypeName='Answer'
EXCEPT
SELECT users.* FROM users, posts, postTypes WHERE users.userid=posts.ownerUserId and posts.postTypeId=PostTypes.postTypeId and postTypeName='Question'
set statistics time off
set statistics io off

checkpoint
dbcc dropcleanbuffers
set statistics io on
set statistics time on
SELECT * FROM users
WHERE EXISTS (
    SELECT 1 FROM posts, PostTypes WHERE posts.postTypeId=PostTypes.postTypeId and postTypeName='Answer' and posts.ownerUserId = users.userId
)
AND NOT EXISTS (
    SELECT 1 FROM posts, PostTypes WHERE posts.postTypeId=PostTypes.postTypeId and postTypeName='Question' and posts.ownerUserId = users.userId
)
set statistics time off
set statistics io off
```



```

checkpoint
dbcc dropcleanbuffers
set statistics io on
set statistics time on
SELECT users.* FROM users, posts, postTypes WHERE users.userid=posts.ownerUserId and posts.postTypeId=PostTypes.postTypeId and postTypeName='Answer'
EXCEPT
SELECT users.* FROM users, posts, postTypes WHERE users.userid=posts.ownerUserId and posts.postTypeId=PostTypes.postTypeId and postTypeName='Question'
set statistics time off
set statistics io off

checkpoint
dbcc dropcleanbuffers
set statistics io on
set statistics time on
SELECT * FROM users WHERE EXISTS (
    SELECT 1 FROM posts, PostTypes WHERE posts.postTypeId=PostTypes.postTypeId and postTypeName='Answer' and posts.ownerUserId = users.userId)
AND NOT EXISTS (
    SELECT 1 FROM posts, PostTypes WHERE posts.postTypeId=PostTypes.postTypeId and postTypeName='Question' and posts.ownerUserId = users.userId)
set statistics time off
set statistics io off

checkpoint
dbcc dropcleanbuffers
create index idx1 on posts(ownerUserId, postTypeId)
set statistics io on
set statistics time on
SELECT * FROM users WHERE EXISTS (
    SELECT 1 FROM posts, PostTypes WHERE posts.postTypeId=PostTypes.postTypeId and postTypeName='Answer' and posts.ownerUserId = users.userId)
AND NOT EXISTS (
    SELECT 1 FROM posts, PostTypes WHERE posts.postTypeId=PostTypes.postTypeId and postTypeName='Question' and posts.ownerUserId = users.userId)
set statistics time off
set statistics io off
drop index idx1 on posts

```

Μετά από δοκιμές, το μόνο ευρετήριο που επίφερε βελτίωση είναι αυτό του παραπάνω στιγμιότυπου στον πίνακα posts για την σύνδεση των id των χρηστών και αυτό γιατί οι πίνακες users και posts, που είναι κρίσιμοι για αυτό το query, είναι και πάρα πολύ μεγάλοι οπότε δημιουργώντας το idx1 ο sql server μπορεί ταχύτατα να εντοπίσει και να κάνει την σύνδεση των post ανά χρήστη οπότε σε συνδυασμό με την exist και not exist πλέον και κοιτάμε μόνο 1 post ανά postTypeName αλλά και πολύ γρήγορα χάρης στο ευρετήριο αυτό. Παρακάτω φαίνεται η σημαντική βελτίωση σε cpu time και elapsed time καθώς και η ραγδαία μείωση των Logical reads στον πίνακα Posts λόγω του ευρετηρίου.

```

Results Messages Execution plan
(70698 rows affected)
Table 'Users'. Scan count 26, logical reads 12002, physical reads 2, page server reads 0, read-ahead reads 5717, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'Posts'. Scan count 26, logical reads 747084, physical reads 2, page server reads 0, read-ahead reads 368368, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'PostTypes'. Scan count 17, logical reads 8, physical reads 1, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
(1 row affected)

SQL Server Execution Times:
    CPU time = 4282 ms,  elapsed time = 2768 ms.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.

(70698 rows affected)
Table 'Users'. Scan count 13, logical reads 6001, physical reads 2, page server reads 0, read-ahead reads 5717, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'Posts'. Scan count 26, logical reads 747064, physical reads 2, page server reads 0, read-ahead reads 368369, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'PostTypes'. Scan count 17, logical reads 8, physical reads 1, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
(1 row affected)

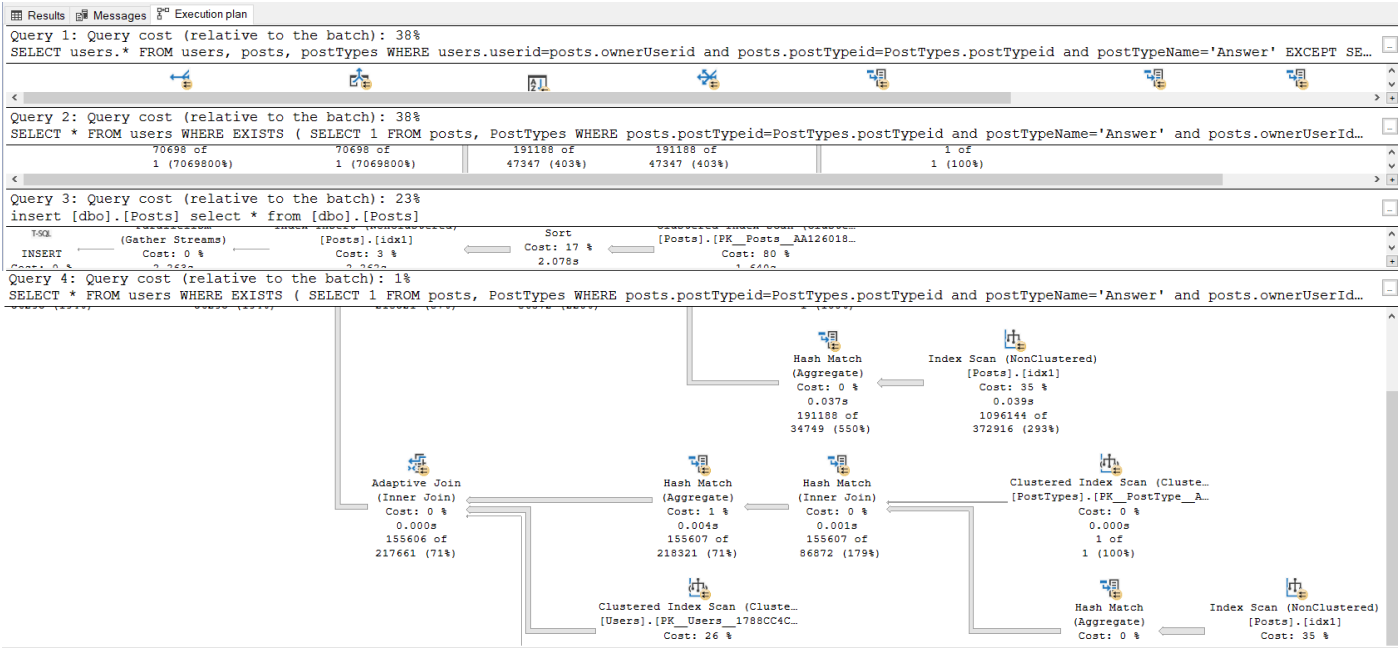
SQL Server Execution Times:
    CPU time = 2939 ms,  elapsed time = 2612 ms.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
SQL Server parse and compile time:
    CPU time = 29 ms, elapsed time = 29 ms.

(70698 rows affected)
Table 'Users'. Scan count 13, logical reads 6001, physical reads 2, page server reads 0, read-ahead reads 5717, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'Posts'. Scan count 26, logical reads 14090, physical reads 0, page server reads 0, read-ahead reads 55, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'PostTypes'. Scan count 21, logical reads 8, physical reads 1, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0.
(1 row affected)

SQL Server Execution Times:
    CPU time = 1045 ms,  elapsed time = 821 ms.

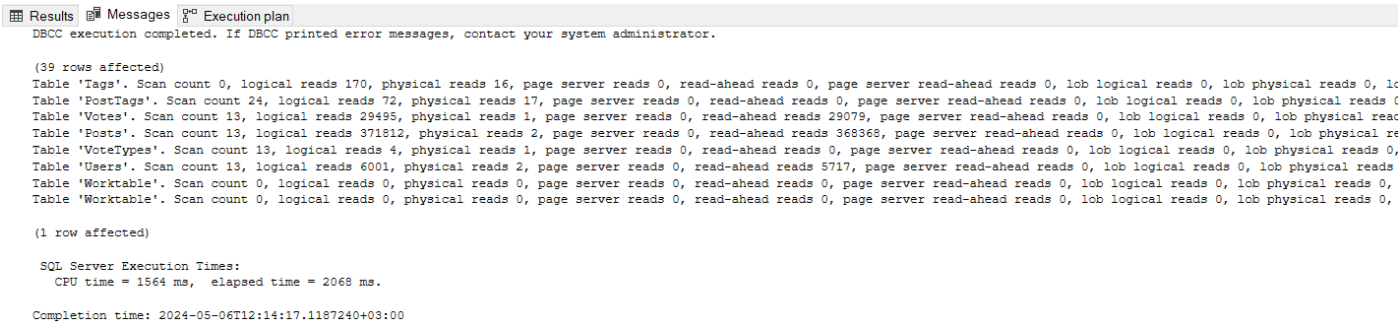
```

Το ποσοστό % του συνολικού κόστους στο query με το ευρετήριο (query 4) βλέπουμε ότι είναι εξαιρετικά χαμηλό κάτι που μας υποδεικνύει ότι έχει βελτιωθεί απόλυτα το query μας από τις αλλαγές του except και του ευρετηρίου που προσθέσαμε, το οποίο παρατηρείται από το παρακάτω στιγμιότυπα του execution plan.

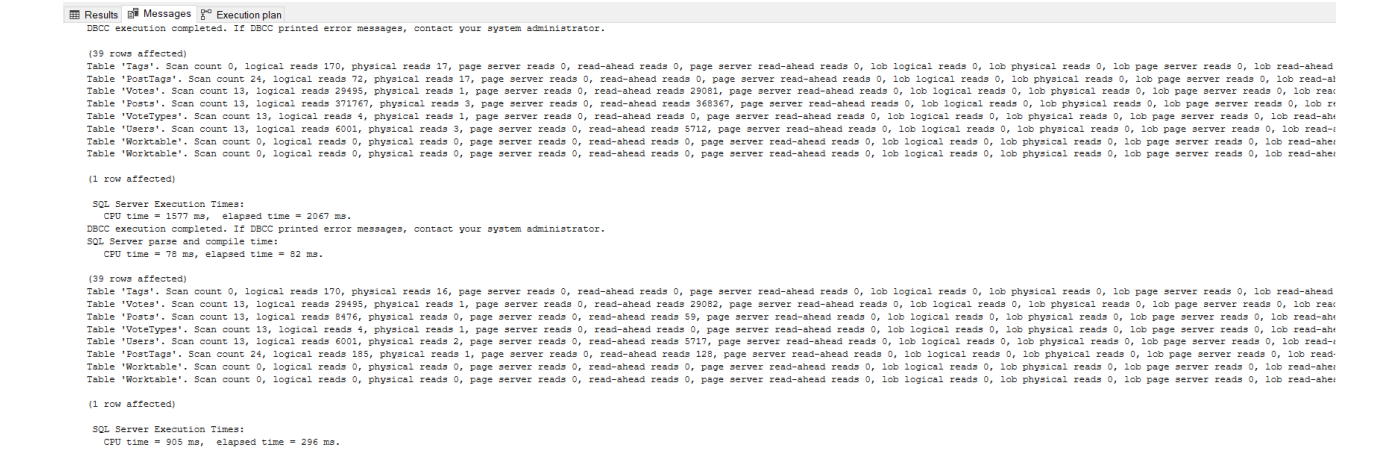


Ζήτημα 3

Αρχικά ας εξετάσουμε τα στατιστικά στοιχεία χρόνου εκτέλεσης και δίσκου



Παρατηρούμε πολλαπλά reads στους πίνακες votes, posts και users, οπότε το πιθανότερο είναι να χρησιμοποιηθούν ευρετήρια προς αυτή την κατεύθυνση για αρχή. Πράγματι, εφαρμόζοντας ευρετήριο αρχικά στον posts στα πεδία που χρησιμοποιεί για inner join έχουμε σημαντική βελτίωση όπως φαίνεται στα παρακάτω στιγμιότυπα

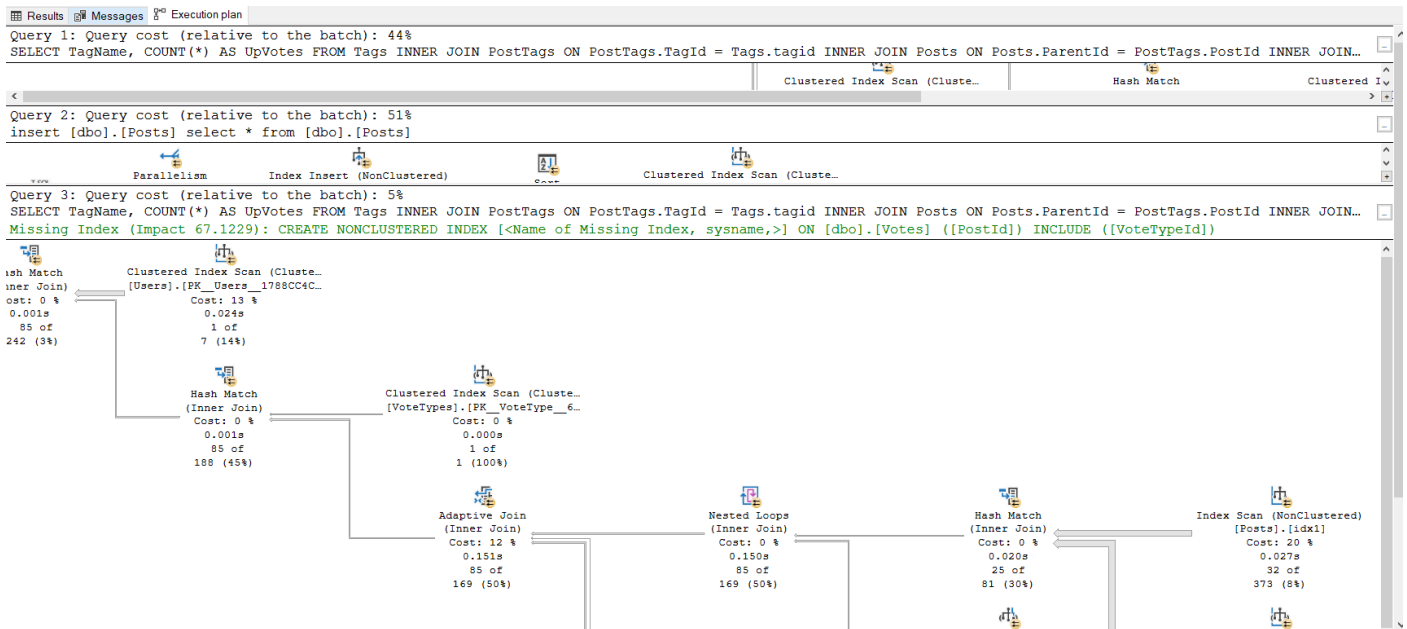


```

checkpoint
dbcc dropcleanbuffers
set statistics io on
set statistics time on
--SELECT TagName, COUNT(*) AS UpVotes FROM Tags
--INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
--INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
--INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
--INNER JOIN Votes ON Votes.PostId = Posts.postId
--INNER JOIN VoteTypes ON Votes.voteTypeId=VoteTypes.VoteTypeID
--WHERE VoteTypeName='UpVote' AND Users.displayName = 'Dominik Weber'
--GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off

checkpoint
dbcc dropcleanbuffers
create index idx1 on Posts(ParentId, OwnerUserId)
set statistics io on
set statistics time on
--SELECT TagName, COUNT(*) AS UpVotes FROM Tags
--INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
--INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
--INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
--INNER JOIN Votes ON Votes.PostId = Posts.postId
--INNER JOIN VoteTypes ON Votes.voteTypeId=VoteTypes.VoteTypeID
--WHERE VoteTypeName='UpVote' AND Users.displayName = 'Dominik Weber'
--GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off
drop index idx1 on Posts

```



Το ευρετήριο idx1 χρησιμοποιείται όπως βλέπουμε στο παραπάνω execution plan, μειώνοντας, σύμφωνα με τα statistics, σημαντικά τον χρόνο εκτέλεσης και τα logical reads του πίνακα Posts. Ύστερα δημιουργούμε το ευρετήριο idx2 στον πίνακα Votes αντίστοιχα σαν του Posts και παρατηρούμε και εδώ περαιτέρω βελτίωση του query μας.

```

checkpoint
dbcc dropcleanbuffers
create index idx1 on Posts(ParentId, OwnerUserId)
set statistics io on
set statistics time on
--SELECT TagName, COUNT(*) AS UpVotes FROM Tags
--INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
--INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
--INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
--INNER JOIN Votes ON Votes.PostId = Posts.postId
--INNER JOIN VoteTypes ON Votes.voteTypeId=VoteTypes.VoteTypeID
--WHERE VoteTypeName='UpVote' AND Users.displayName = 'Dominik Weber'
--GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off
drop index idx1 on Posts

checkpoint
dbcc dropcleanbuffers
create index idx1 on Posts(ParentId, OwnerUserId)
create index idx2 on Votes(PostId, voteTypeId)
set statistics io on
set statistics time on
--SELECT TagName, COUNT(*) AS UpVotes FROM Tags
--INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
--INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
--INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
--INNER JOIN Votes ON Votes.PostId = Posts.postId
--INNER JOIN VoteTypes ON Votes.voteTypeId=VoteTypes.VoteTypeID
--WHERE VoteTypeName='UpVote' AND Users.displayName = 'Dominik Weber'
--GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off
drop index idx1 on Posts
drop index idx2 on Votes

```


Results

Messages

Execution plan

```
SQL Server parse and compile time:
    CPU time = 86 ms, elapsed time = 86 ms.
```

[illegible]

```
(1 row affected)
```

SQL Server Execution Times:
CPU time = 1109 ms, elapsed time = 355 ms.

DBCC execution completed. If DBCC printed error messages, contact your system administrator.
SQL Server parse and compile time:
CPU time = 47 ms, elapsed time = 48 ms.

```

(93 rows affected)
Table 'Tags'. Scan count 0, logical reads 170, physical reads 37, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob re
Table 'Votes'. Scan count 106, logical reads 318, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob re
Table 'PostTags'. Scan count 31, logical reads 93, physical reads 46, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob re
Table 'Posts'. Scan count 13, logical reads 8476, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob re
Table 'Users'. Scan count 13, logical reads 6001, physical reads 3, page server reads 0, read-ahead reads 5712, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob re
Table 'VoteTypes'. Scan count 13, logical reads 4, physical reads 1, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob re
Table 'Comments'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob re
Table 'Workable'. Scan count 0, logical reads 0, physical reads page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob re

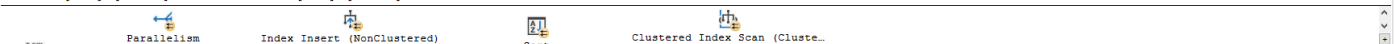
```

(1 row affected)

SQL Server Execution Times:
CPU time = 451 ms, elapsed time = 174 ms.

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 38%
insert [dbo].[Posts] select * from [dbo].[Posts]

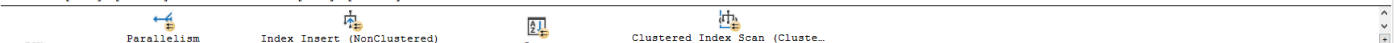


Query 2: Query cost (relative to the batch): 4%

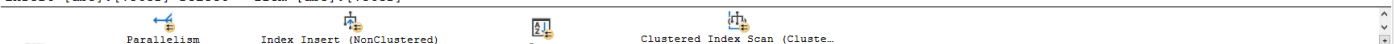
```
SELECT TagName, COUNT(*) AS UpVotes FROM Tags INNER JOIN PostTags ON PostTags.TagId = Tags.tagid INNER JOIN Posts ON Posts.ParentId = PostTags.PostId INNER JOIN Us...  
Missing Index (Impact 67.1229): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [dbo].[Votes] ([PostId]) INCLUDE ([VoteTypeId])
```

Query 3: Query cost (relative to the batch): 38%

```
insert [dbo].[Posts] select * from [dbo].[Posts]
```

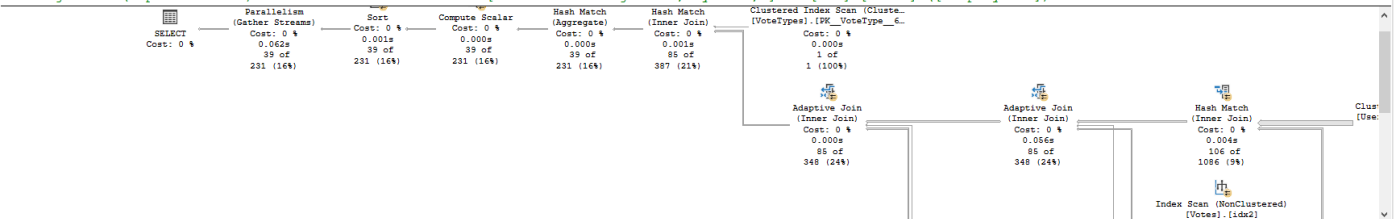


```
Query 4: Query cost (relative to the batch): 19%
insert [dbo].[Votes] select * from [dbo].[Votes]
```



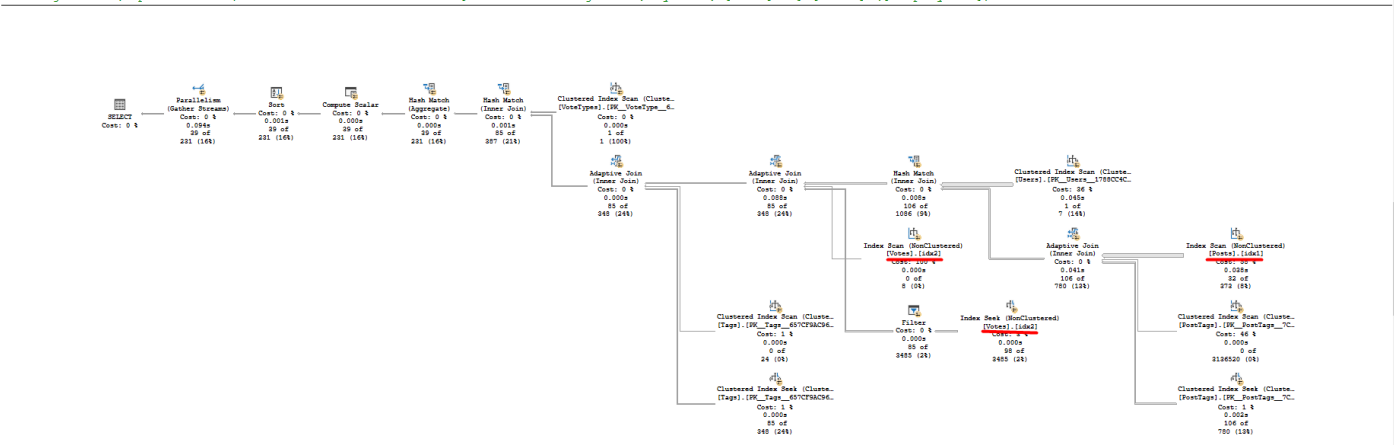
Query 5: Query cost (relative to the batch): **1%**

```
SELECT TagName, COUNT(*) AS UpVotes FROM Tags INNER JOIN PostTags ON PostTags.TagId = Tags.tagid INNER JOIN Posts ON Posts.ParentId = PostTags.PostId INNER JOIN Us...
Missing Index (Impact 36.5812): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [dbo].[Users] ([DisplayName])
```



Query 1: με ευρετήριο idx1, Query 5: με ευρετήρια idx1 & idx2

```
Query 5: Query cost (relative to the batch): 1%
SELECT TagName, COUNT(*) AS UpVotes FROM Tags INNER JOIN PostTags ON PostTags.TagId = Tags.TagId INNER JOIN Posts ON Posts.ParentId = PostTags.PostId INNER JOIN Users ON Pos...
```



```
checkpoint
dbcc dropcleanbuffers

create index idx1_1 on Posts(ParentId, OwnerUserId)
create index idx2 on Votes(PostId, voteTypeId)
set statistics io on
set statistics time on
SELECT TagName, COUNT(*) AS UpVotes FROM Tags
INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
INNER JOIN Votes ON Votes.PostId = Posts.postid
INNER JOIN VoteTypes ON Votes.voteTypeId=VoteTypes.VoteTypeId
WHERE VoteTypeName='UpVote' AND Users.displayName = 'Dominik Weber'
GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off
drop index idx1_1 on Posts
drop index idx2 on Votes

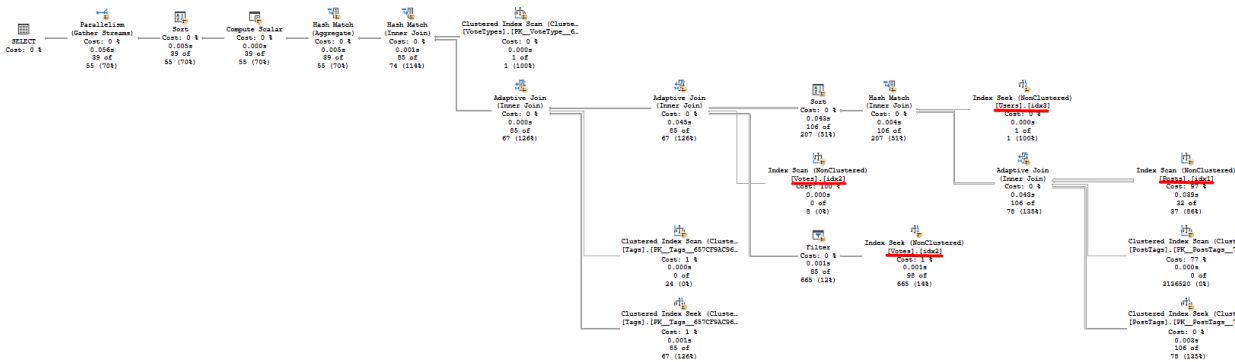
checkpoint
dbcc dropcleanbuffers

create index idx1_2 on Posts(ParentId, OwnerUserId)
create index idx2_1 on Votes(PostId, voteTypeId)
create index idx3 on Users(displayName)
set statistics io on
set statistics time on
SELECT TagName, COUNT(*) AS UpVotes FROM Tags
INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
INNER JOIN Votes ON Votes.PostId = Posts.postid
INNER JOIN VoteTypes ON Votes.voteTypeId=VoteTypes.VoteTypeId
WHERE VoteTypeName='UpVote' AND Users.displayName = 'Dominik Weber'
GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off
drop index idx1_2 on Posts
drop index idx2_1 on Votes
drop index idx3 on Users
```

Query 1: ευρετήρια idx1 & idx2, Query 7: Ευρετήρια idx1, idx2 & idx3

[illegible]

Query 7: Query cost (relative to the batch): 1%
SELECT TagName, COUNT(*) AS UpVotes FROM Tags INNER JOIN PostTags ON PostTags.TagId = Tags.tagid INNER JOIN Posts ON Posts.ParentId = PostTags.PostId INNER JOIN Users ON Pos...



Πρέπει να σημειωθεί πως δοκιμάστηκε ευρετήριο για τον πίνακα PostTags (create index idx4 on PostTags(TagId)) για να μειώσουμε το 77% που βλέπουμε στο execution plan όπου χρησιμοποιήθηκε το ευρετήριο από τον sql server αλλά δεν επέφερε καμία αλλαγή σε χρόνους εκτέλεσης, logical reads και total query batch cost.

Παρακάτω συγκρίνουμε όλα τα queries (αρχικό, με idx1, με ldx1 & idx2, με idx1, idx2 & idx3)

```
--checkpoint
dbcc dropcleanbuffers
set statistics io on
set statistics time on
--SELECT TagName, COUNT(*) AS UpVotes FROM Tags
INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
INNER JOIN Votes ON Votes.PostId = Posts.postid
INNER JOIN Votetypes ON Votes.votepid=Votetypes.Votepid
WHERE VotepName='UpVote' AND Users.displayName = 'Dominik Weber'
GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off

checkpoint
dbcc dropcleanbuffers
create index idx1 on Posts(ParentId, OwnerUserId)
set statistics io on
set statistics time on
--SELECT TagName, COUNT(*) AS UpVotes FROM Tags
INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
INNER JOIN Votes ON Votes.PostId = Posts.postid
INNER JOIN Votetypes ON Votes.votepid=Votetypes.Votepid
WHERE VotepName='UpVote' AND Users.displayName = 'Dominik Weber'
GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off

checkpoint
dbcc dropcleanbuffers
create index idx2 on Votes(Postid, votepid)
set statistics io on
set statistics time on
--SELECT TagName, COUNT(*) AS UpVotes FROM Tags
INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
INNER JOIN Votes ON Votes.PostId = Posts.postid
INNER JOIN Votetypes ON Votes.votepid=Votetypes.Votepid
WHERE VotepName='UpVote' AND Users.displayName = 'Dominik Weber'
GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off

checkpoint
dbcc dropcleanbuffers
create index ldx1 on Users(displayName)
set statistics io on
set statistics time on
--SELECT TagName, COUNT(*) AS UpVotes FROM Tags
INNER JOIN PostTags ON PostTags.TagId = Tags.tagid
INNER JOIN Posts ON Posts.ParentId = PostTags.PostId
INNER JOIN Users ON Posts.OwnerUserId=Users.UserId
INNER JOIN Votes ON Votes.PostId = Posts.postid
INNER JOIN Votetypes ON Votes.votepid=Votetypes.Votepid
WHERE VotepName='UpVote' AND Users.displayName = 'Dominik Weber'
GROUP BY TagName ORDER BY UpVotes DESC
set statistics time off
set statistics io off
drop index idx1 on Posts
drop index idx2 on Votes
drop index idx3 on Users
```


Αρχικά ας εκτελέσουμε το query χωρίς ευρετήρια

Δημιουργούμε τα ευρετήρια idx1 και idx2 που φαίνονται παρακάτω

```

Results Messages Execution plan
DBCC execution completed. If DBCC printed error messages, contact your system administrator.

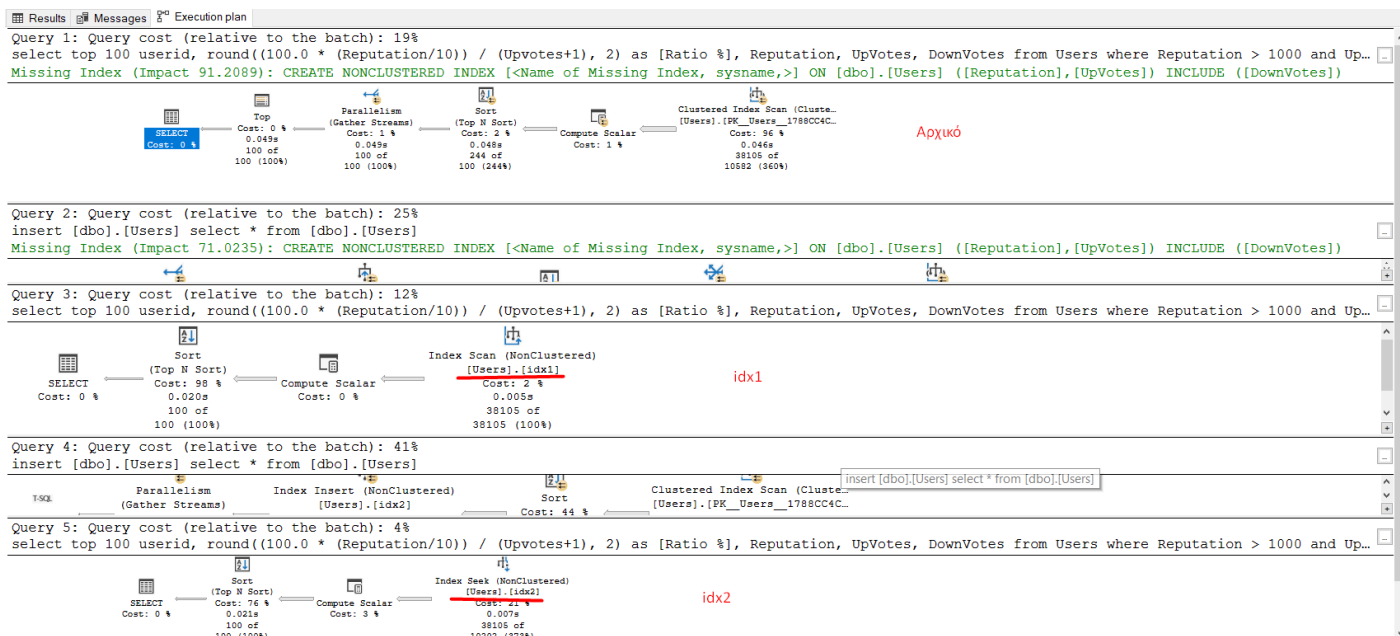
(100 rows affected)
Table 'Users'. Scan count 13, logical reads 6001, physical reads 2, page server reads 0, read-ahead reads 5717, page server read-ahead reads 0, lob logical reads 0, lob physica
(1 row affected)
SQL Server Execution Times:
    CPU time = 15 ms, elapsed time = 63 ms.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
SQL Server parse and compile time:
    CPU time = 0 ms, elapsed time = 4 ms.

(100 rows affected)
Table 'Users'. Scan count 1, logical reads 112, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical
(1 row affected)
SQL Server Execution Times:
    CPU time = 16 ms, elapsed time = 84 ms.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
SQL Server parse and compile time:
    CPU time = 2 ms, elapsed time = 2 ms.

(100 rows affected)
Table 'Users'. Scan count 1, logical reads 157, physical reads 0, page server reads 0, read-ahead reads 7, page server read-ahead reads 0, lob logical reads 0, lob physical
(1 row affected)
SQL Server Execution Times:
    CPU time = 16 ms, elapsed time = 80 ms.

Completion time: 2024-05-06T14:54:51.0991721+03:00

```



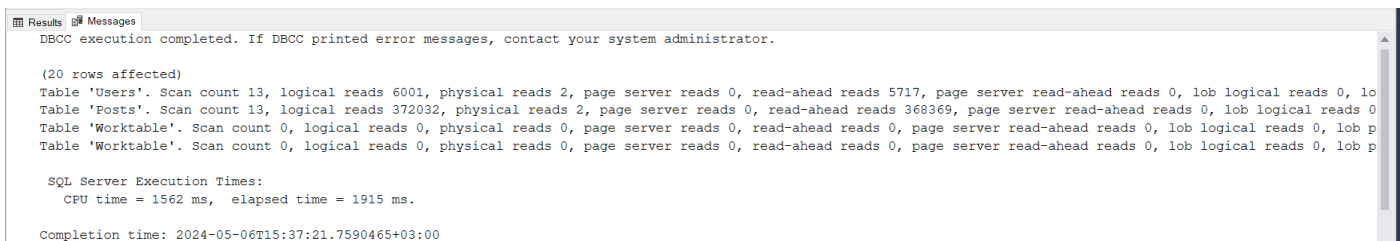
Αν και το idx2 έχει μικρότερο query batch cost (4%) τα logical reads του είναι περισσότερα από το idx1. Αυτό μπορεί να συμβεί διότι οι μετρήσεις του sql server στο execution plan είναι εκτιμήσεις που βασίζονται σε στατιστικά στοιχεία και μπορεί να έχουν απόκλιση από την πραγματική απόδοση. Το idx1 έχει λιγότερα Logical reads και είναι κατασκευασμένο ακριβώς για τα δεδομένα του query που έχουμε, συγκεκριμενοποιημένο παρέχοντας μικρότερο χώρο αναζήτησης και πιο στοχευμένο από το idx2 οπότε ταιριάζει περισσότερο στην περίπτωση μας. Μπορεί να μην φαίνεται στο execution plan λόγω του πλήθους των δεδομένων που έχουμε αλλά σίγουρα σε μεγαλύτερο όγκο δεδομένων το idx1 είναι το ιδανικό για αυτό το query καθώς θα παρέχει πάντα το μικρότερο δυνατό σύνολο δεδομένων αναζήτησης για την μέγιστη απόδοση και ταχύτητα του query. Εν κατακλείδι επιλέγουμε το idx1.

Ζήτημα 5

1. Επιλέγουμε τους top 20 χρήστες, οι οποίοι έχουν κάνει τουλάχιστον 100 αναρτήσεις σε φθίνουσα σειρά βάση του μέσου σκορ των αναρτήσεών τους.

	Results	Messages	
	DisplayName	UserId	Avg_Score
1	Readonly	4883	183
2	Ryan McGeary	8985	151
3	e-satis	9951	130
4	venkatachalam	44984	95
5	Dan Moulding	95706	94
6	Hamza Yerlikaya	89904	93
7	Pat Notz	825	90
8	Fabien Ménager	92315	89
9	Esko Luontola	62130	83
10	Daryl Spitzer	4766	82
11	Ray Vega	4872	75
12	Matt Sheppard	797	73
13	Grundfeleck	4120	71
14	Blair Conrad	1199	68
15	jop	11830	65
16	pupeno	6068	63
17	Derek Park	872	63
18	Sergio del Arno	2138	61
19	Jeff Atwood	1	61
20	Aristotle Pagaltzis	9410	58

```
select top 20
    DisplayName, UserId, AVG(Posts.Score) as Avg_Score
from Users
inner join Posts on UserId = Posts.OwnerUserId
group by DisplayName, UserId
having count(Posts.PostId) > = 100
order by Avg Score desc
```



2. Δημιουργούμε τα ευρετήρια idx1 & idx2

```
checkpoint
dbcc dropcleanbuffers
set statistics io on
set statistics time on
select top 20
    DisplayName, UserId, AVG(Posts.Score) as Avg_Score
from Users
inner join Posts on UserId = Posts.OwnerUserId
group by DisplayName, UserId
having count(Posts.PostId) >= 100
order by Avg_Score desc
set statistics time off
set statistics io off

checkpoint
dbcc dropcleanbuffers
create index idx1 on Posts (OwnerUserId) include (Score, PostId)
create index idx2 on Users (UserId) include (DisplayName)
set statistics io on
set statistics time on
select top 20
    DisplayName, UserId, AVG(Posts.Score) as Avg_Score
from Users
inner join Posts on UserId = Posts.OwnerUserId
group by DisplayName, UserId
having count(Posts.PostId) >= 100
order by Avg_Score desc
set statistics time off
set statistics io off
drop index idx1 on Posts
drop index idx2 on Users
```

Results Messages Execution plan

DBCC execution completed. If DBCC printed error messages, contact your system administrator.

(20 rows affected)

Table 'Users'. Scan count 13, logical reads 6001, physical reads 2, page server reads 0, read-ahead reads 5717, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob

Table 'Posts'. Scan count 13, logical reads 5717, physical reads 2, page server reads 0, read-ahead reads 36596, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob

(1 row affected)

SQL Server Execution Times:
CPU time = 1750 ms, elapsed time = 2116 ms.

DBCC execution completed. If DBCC printed error messages, contact your system administrator.

SQL Server parse and compile time:
CPU time = 8 ms, elapsed time = 8 ms.

(20 rows affected)

Table 'Posts'. Scan count 13, logical reads 8484, physical reads 0, page server reads 0, read-ahead reads 49, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob

Table 'Users'. Scan count 13, logical reads 1195, physical reads 0, page server reads 0, read-ahead reads 25, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob

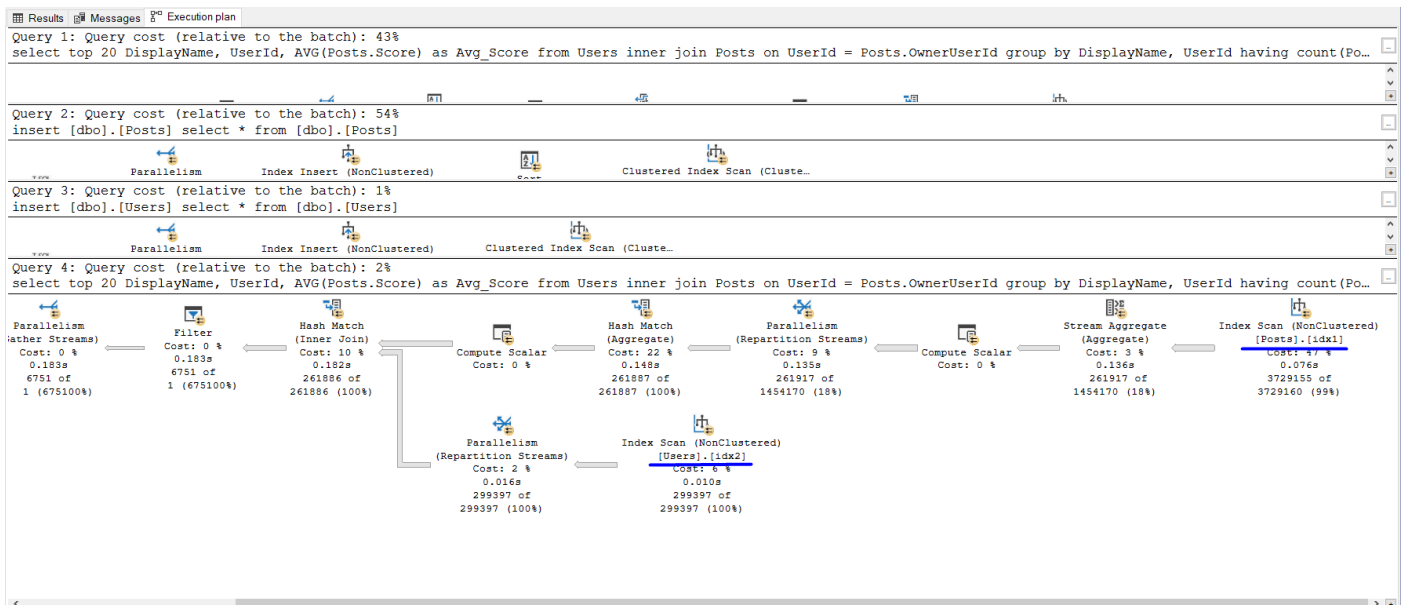
(1 row affected)

SQL Server Execution Times:
CPU time = 1907 ms, elapsed time = 265 ms.

Completion time: 2024-05-06T16:13:52.6649057+03:00

Αρχικό

idx1_idx2



Το idx1 μας εξασφαλίζει ταχεία λειτουργία του inner join και περιλαμβάνει το column score για γρήγορο υπολογισμό του avg και το column PostId για ευκολότερη και γρηγορότερη καταμέτρηση των αναρτήσεων. Το idx2 εξασφαλίζει άμεση ανάκτηση των displaynames βάση των UserId για γρηγορότερη εκτέλεση και εμφάνιση τελικών αποτελεσμάτων αλλά και για την ταχεία λειτουργία του inner join όπως και στο idx1 (UserId = Posts.OwnerUserId)